Swap chain recreation

Introduction

The application we have now successfully draws a triangle, but there are some circumstances that it isn't handling properly yet. It is possible for the window surface to change such that the swap chain is no longer compatible with it. One of the reasons that could cause this to happen is the size of the window changing. We have to catch these events and recreate the swap chain.

Recreating the swap chain

Create a new `recreateSwapChain` function that calls `createSwapChain` and all of the creation functions for the objects that depend on the swap chain or the window size.

```
void recreateSwapChain() {
    vkDeviceWaitIdle(device);

    createSwapChain();
    createImageViews();
    createFramebuffers();
}
```

We first call `vkDeviceWaitIdle` (https://www.khronos.org/registry/vulkan/specs/1.0/man/html/vkDeviceWaitIdle.html), because just like in the last chapter, we shouldn't touch resources that may still be in use. Obviously, we'll have to recreate the swap chain itself. The image views need to be recreated because they are based directly on the swap chain images. Finally, the framebuffers directly depend on the swap chain images, and thus must be recreated as well.

To make sure that the old versions of these objects are cleaned up before recreating them, we should move some of the cleanup code to a separate function that we can call from the `recreateSwapChain` function. Let's call it `cleanupSwapChain`:

```
void cleanupSwapChain() {

}

void recreateSwapChain() {
    vkDeviceWaitIdle(device);

    cleanupSwapChain();

    createSwapChain();
    createImageViews();
    createFramebuffers();
}
```

Note that we don't recreate the renderpass here for simplicity. In theory it can be possible for the swap chain image format to change during an applications' lifetime, e.g. when moving a window from a standard range to a high dynamic range monitor. This may require the application to recreate the renderpass to make sure the change between dynamic ranges is properly reflected.

We'll move the cleanup code of all objects that are recreated as part of a swap chain refresh from `cleanup` to `cleanupSwapChain`:

```
void cleanupSwapChain() {
    for (auto framebuffer : swapChainFramebuffers) {
        vkDestroyFramebuffer(device, framebuffer, nullptr);
    }

    for (auto imageView : swapChainImageViews) {
        vkDestroyImageView(device, imageView, nullptr);
    }

    vkDestroySwapchainKHR(device, swapChain, nullptr);
}

void cleanup() {
    cleanupSwapChain();

    vkDestroyPipeline(device, graphicsPipeline, nullptr);
    vkDestroyPipelineLayout(device, pipelineLayout, nullptr);

    vkDestroyRenderPass(device, renderPass, nullptr);

    for (size_t i = 0; i < MAX_FRAMES_IN_FLIGHT; i++) {
        vkDestroySemaphore(device, renderFinishedSemaphores[i], nullptr);
        vkDestroySemaphore(device, imageAvailableSemaphores[i], nullptr);
        vkDestroyFence(device, inFlightFences[i], nullptr);
    }

    vkDestroyCommandPool(device, commandPool, nullptr);

    vkDestroyDevice(device, nullptr);

    if (enableValidationLayers) {
        DestroyDebugUtilsMessengerEXT(instance, debugMessenger, nullptr);
    }

    vkDestroySurfaceKHR(instance, surface, nullptr);
    vkDestroyInstance(instance, nullptr);

    glfwDestroyWindow(window);

    glfwTerminate();
}
```

Note that in `chooseSwapExtent` we already query the new window resolution to make sure that the swap chain images have the (new) right size, so there's no need to modify `chooseSwapExtent` (remember that we already had to use `glfwGetFramebufferSize` to get the resolution of the surface in pixels when creating the swap chain).

That's all it takes to recreate the swap chain! However, the disadvantage of this approach is that we need to stop all rendering before creating the new swap chain. It is possible to create a new swap chain while drawing commands on an image from the old swap chain are still in-flight. You need to pass the previous swap chain to the `oldSwapChain` field in the `VkSwapchainCreateInfoKHR` struct and destroy the old swap

chain as soon as you've finished using it.

Suboptimal or out-of-date swap chain

Now we just need to figure out when swap chain recreation is necessary and call our new `recreateSwapChain` function. Luckily, Vulkan will usually just tell us that the swap chain is no longer adequate during presentation. The `vkAcquireNextImageKHR` and `vkQueuePresentKHR` functions can return the following special values to indicate this.

- `VK_ERROR_OUT_OF_DATE_KHR`: The swap chain has become incompatible with the surface and can no longer be used for rendering. Usually happens after a window resize.
- `VK_SUBOPTIMAL_KHR`: The swap chain can still be used to successfully present to the surface, but the surface properties are no longer matched exactly.

```
VkResult result = vkAcquireNextImageKHR(device, swapChain, UINT64_MAX, imageAvailableSe
maphores[currentFrame], VK_NULL_HANDLE, &imageIndex);

if (result == VK_ERROR_OUT_OF_DATE_KHR) {
    recreateSwapChain();
    return;
} else if (result != VK_SUCCESS && result != VK_SUBOPTIMAL_KHR) {
    throw std::runtime_error("failed to acquire swap chain image!");
}
```

If the swap chain turns out to be out of date when attempting to acquire an image, then it is no longer possible to present to it. Therefore we should immediately recreate the swap chain and try again in the next `drawFrame` call.

You could also decide to do that if the swap chain is suboptimal, but I've chosen to proceed anyway in that case because we've already acquired an image. Both `VK_SUCCESS` and `VK_SUBOPTIMAL_KHR` are considered "success" return codes.

```
result = vkQueuePresentKHR(presentQueue, &presentInfo);

if (result == VK_ERROR_OUT_OF_DATE_KHR || result == VK_SUBOPTIMAL_KHR) {
    recreateSwapChain();
} else if (result != VK_SUCCESS) {
    throw std::runtime_error("failed to present swap chain image!");
}

currentFrame = (currentFrame + 1) % MAX_FRAMES_IN_FLIGHT;
```

The `vkQueuePresentKHR` function returns the same values with the same meaning. In this case we will also recreate the swap chain if it is suboptimal, because we want the best possible result.

Fixing a deadlock

If we try to run the code now, it is possible to encounter a deadlock. Debugging the code, we find that the application reaches `vkWaitForFences` (https://www.khronos.org/registry/vulkan/specs/1.0/man/html/

vkWaitForFences.html) but never continues past it. This is because when `vkAcquireNextImageKHR` returns `VK_ERROR_OUT_OF_DATE_KHR`, we recreate the swapchain and then return from `drawFrame`. But before that happens, the current frame's fence was waited upon and reset. Since we return immediately, no work is submitted for execution and the fence will never be signaled, causing `vkWaitForFences` (https://www.khronos.org/registry/vulkan/specs/1.0/man/html/vkWaitForFences.html) to halt forever.

There is a simple fix thankfully. Delay resetting the fence until after we know for sure we will be submitting work with it. Thus, if we return early, the fence is still signaled and `vkWaitForFences` (https://www.khronos.org/registry/vulkan/specs/1.0/man/html/vkWaitForFences.html) wont deadlock the next time we use the same fence object.

The beginning of `drawFrame` should now look like this:

```
vkWaitForFences(device, 1, &inFlightFences[currentFrame], VK_TRUE, UINT64_MAX);

uint32_t imageIndex;
VkResult result = vkAcquireNextImageKHR(device, swapChain, UINT64_MAX, imageAvailableSe
maphores[currentFrame], VK_NULL_HANDLE, &imageIndex);

if (result == VK_ERROR_OUT_OF_DATE_KHR) {
    recreateSwapChain();
    return;
} else if (result != VK_SUCCESS && result != VK_SUBOPTIMAL_KHR) {
    throw std::runtime_error("failed to acquire swap chain image!");
}

// Only reset the fence if we are submitting work
vkResetFences(device, 1, &inFlightFences[currentFrame]);
```

Handling resizes explicitly

Although many drivers and platforms trigger `VK_ERROR_OUT_OF_DATE_KHR` automatically after a window resize, it is not guaranteed to happen. That's why we'll add some extra code to also handle resizes explicitly. First add a new member variable that flags that a resize has happened:

```
std::vector<VkFence> inFlightFences;

bool framebufferResized = false;
```

The `drawFrame` function should then be modified to also check for this flag:

```
if (result == VK_ERROR_OUT_OF_DATE_KHR || result == VK_SUBOPTIMAL_KHR || framebufferRes
ized) {
    framebufferResized = false;
    recreateSwapChain();
} else if (result != VK_SUCCESS) {
    ...
}
```

It is important to do this after `vkQueuePresentKHR` to ensure that the semaphores are in a consistent state, otherwise a signaled semaphore may never be properly waited upon. Now to actually detect resizes we can use the `glfwSetFramebufferSizeCallback` function in the GLFW framework to set up a callback:

```
void initWindow() {
    glfwInit();

    glfwWindowHint(GLFW_CLIENT_API, GLFW_NO_API);

    window = glfwCreateWindow(WIDTH, HEIGHT, "Vulkan", nullptr, nullptr);
    glfwSetFramebufferSizeCallback(window, framebufferResizeCallback);
}

static void framebufferResizeCallback(GLFWwindow* window, int width, int height) {

}
```

The reason that we're creating a `static` function as a callback is because GLFW does not know how to properly call a member function with the right `this` pointer to our `HelloTriangleApplication` instance.

However, we do get a reference to the `GLFWwindow` in the callback and there is another GLFW function that allows you to store an arbitrary pointer inside of it: `glfwSetWindowUserPointer`:

```
window = glfwCreateWindow(WIDTH, HEIGHT, "Vulkan", nullptr, nullptr);
glfwSetWindowUserPointer(window, this);
glfwSetFramebufferSizeCallback(window, framebufferResizeCallback);
```

This value can now be retrieved from within the callback with `glfwGetWindowUserPointer` to properly set the flag:

```
static void framebufferResizeCallback(GLFWwindow* window, int width, int height) {
    auto app = reinterpret_cast<HelloTriangleApplication*>(glfwGetWindowUserPointer(win
dow));
    app->framebufferResized = true;
}
```

Now try to run the program and resize the window to see if the framebuffer is indeed resized properly with the window.

Handling minimization

There is another case where a swap chain may become out of date and that is a special kind of window resizing: window minimization. This case is special because it will result in a frame buffer size of `0`. In this tutorial we will handle that by pausing until the window is in the foreground again by extending the `recreateSwapChain` function:

```
void recreateSwapChain() {
    int width = 0, height = 0;
    glfwGetFramebufferSize(window, &width, &height);
    while (width == 0 || height == 0) {
        glfwGetFramebufferSize(window, &width, &height);
        glfwWaitEvents();
    }

    vkDeviceWaitIdle(device);

    ...
}
```

The initial call to `glfwGetFramebufferSize` handles the case where the size is already correct and `glfwWaitEvents` would have nothing to wait on.

Congratulations, you've now finished your very first well-behaved Vulkan program! In the next chapter we're going to get rid of the hardcoded vertices in the vertex shader and actually use a vertex buffer.

C++ code (../../code/17_swap_chain_recreation.cpp) / Vertex shader (../../code/09_shader_base.vert) / Fragment shader (../../code/09_shader_base.frag)

Previous (/en/Drawing_a_triangle/Drawing/Frames_in_flight)
Next (/en/Vertex_buffers/Vertex_input_description)